

Índice

1. Análisis de alto nivel
2. Diseño general del sistema y herramientas de sincronización
 - 2.1. Arquitectura y paquetes
 - 2.2. Mecanismos de sincronización
 - 2.3. Flujo del Portal
 - 2.4. Eventos globales aleatorios
 - 2.5. Parte distribuida (RMI)
3. Descripción de las clases principales
4. Diagrama de clases
5. Capturas de la interfaz
6. Anexo: código fuente

1. Análisis de alto nivel

El sistema simula la convivencia entre los niños de Hawkins y los demogorgons del Upside Down segun el enunciado de la asignatura. Existen dos dimensiones interconectadas: Hawkins (zona segura) y el Upside Down (zona insegura donde patrullan los demogorgons). Cuatro portales comunican ambas dimensiones, y una zona especial llamada Colmena guarda a los niños capturados.

Sobre este escenario se generan, de forma escalonada (intervalos aleatorios de 0,5 a 2 segundos), 1.500 hilos Niño. Cada uno repite indefinidamente un ciclo: Sótano Byers → Portal (forma grupo) → Zona insegura → Portal de vuelta → Radio WSQK → Calle Principal. En paralelo, un Demogorgon Alpha (D0000) patrulla las cuatro zonas inseguras desde el inicio; cada vez que la Colmena acumula 8 capturas, Vecna genera un nuevo demogorgon.

El ciclo esta sometido a cuatro eventos globales aleatorios (apagón del laboratorio, tormenta del Upside Down, intervención de Eleven y red mental) que afectan a todos los hilos activos simultáneamente. Toda la actividad se registra en el fichero hawkins.txt y se visualiza en una interfaz Swing que se actualiza en tiempo real.

La Parte 2 del enunciado añade un módulo distribuido vía RMI que publica un servicio (HawkinsRemote) para consultar el estado del sistema y pausar/reanudar la simulacion desde un cliente Swing autónomo.

Actores principales

- Niños (1.500 instancias, escalonadas 0,5–2 s).
- Demogordons (1 inicial D0000, +1 porcada 8 capturas).
- Portales (4): tamaño de grupo 2/3/4/2 y prioridad de retorno.
- Zonas seguras: Calle Principal, Sótano Byers, Radio WSQK.
- Zonas inseguras: Bosque, Laboratorio, Centro Comercial, Alcantarillado, más la Colmena.
- GestorEventos: hilo daemon que dispara los eventos globales.
- SistemaLog: recurso compartido (singleton, lock justo).
- InicioGUI + HawkingsGUI + ClienteRemoto: portada de arranque, GUI local y GUIremota.

2. Diseño general del sistema y herramientas de sincronización

2.1. Arquitectura y paquetes

El proyecto se organiza en seis paquetes con dependencias estrictamente acíclicas. Las clases del modelo (model) y de las zonas (zones) no conocen ni la GUI ni RMI; los paquetes gui y remote dependen del modelo solo en lectura. state y log son transversales.

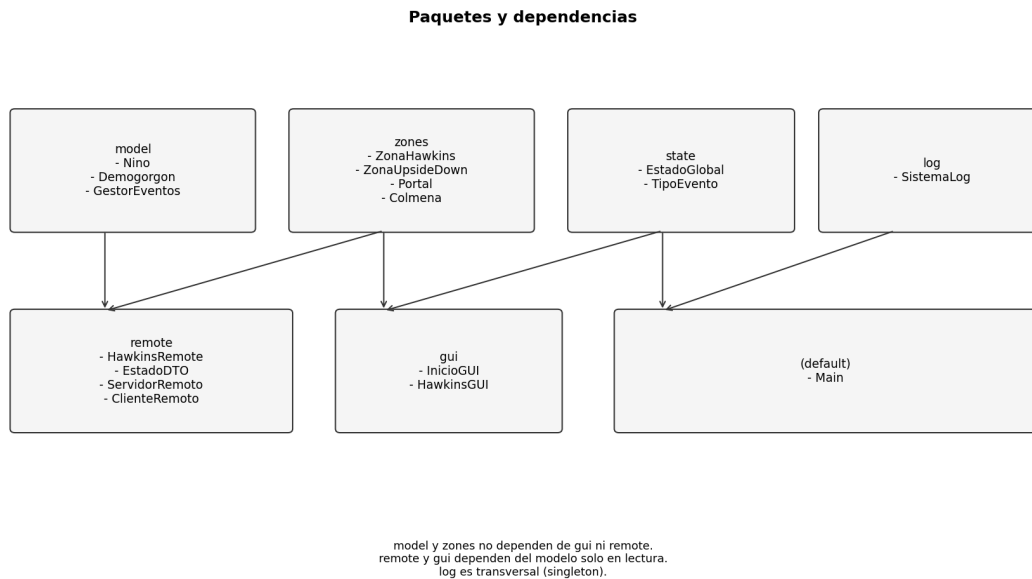


Figura 1. Diagrama de paquetes y dependencias.

Arquitectura general

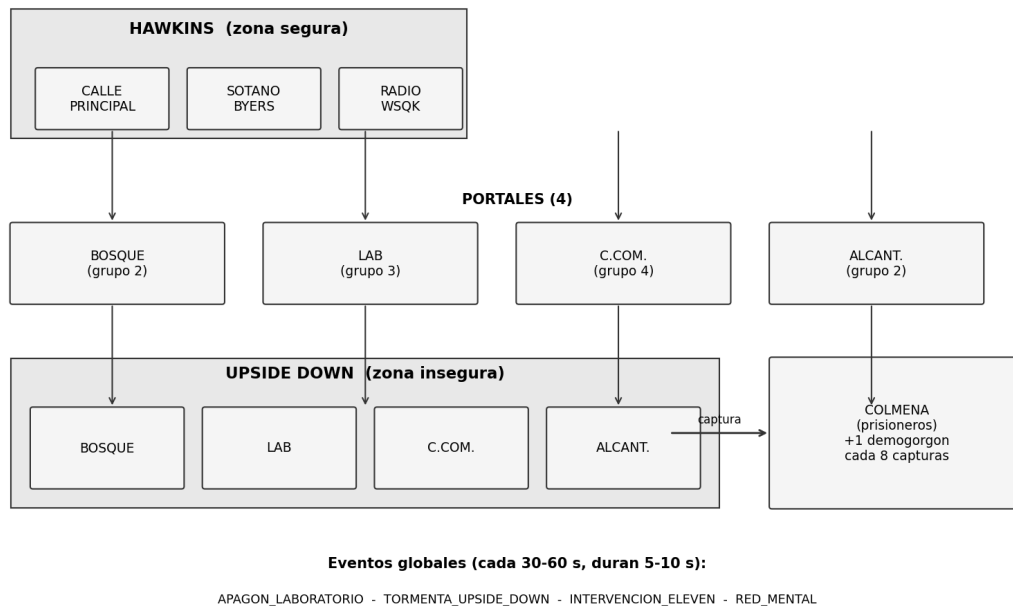


Figura 2. Arquitectura general — zonas, portales, Colmena y eventos.

2.2. Mecanismos de sincronización

Se han elegido los mecanismos mas expresivos y eficientes en cada caso, evitando synchronized implícito cuando hay varias condiciones o prioridades.

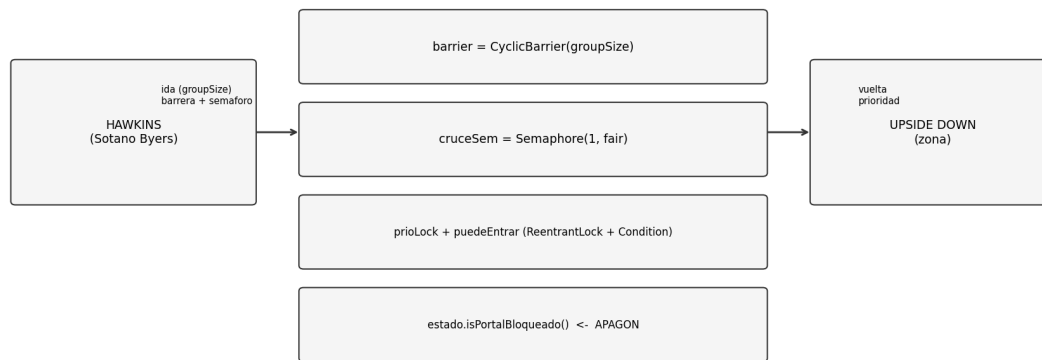
Recurso / Problema	Mecanismo Java	Motivo
Logger (hawkins.txt)	ReentrantLock (fair) + PrintWriter	Mantiene orden FIFO y evita intercalado.
Listas de niños/demos por zona	ReentrantLock (fair) + ArrayList	Operaciones compuestas atómicas para snapshots.
Portal — formación de grupo	CyclicBarrier(groupSize)	Ajuste exacto al tamaño de grupo del enunciado.
Portal — cruce 1 a 1	Semaphore(1, fair)	Garantiza un único transeúnte por portal.
Portal — prioridad de retorno	ReentrantLock + Condition	Las idas esperan mientras retornando > 0.
Portal — apagón	Flag volátil + busy-wait controlado	Bloquea entradas; los que ya cruzan terminan.
Sangre de Vecna	AtomicInteger + CAS	Incrementos y consumo libres de bloqueos.
Colmena (umbral de 8)	synchronized + callback Consumer	Spawn de demogorgons fuera del lock crítico.
Ataque niño ↔ demogorgón	ReentrantLock + Condition	El niño espera a que el demogorgón resuelva.
Liberación por Eleven	ReentrantLock + Condition	El niño capturado duerme hasta liberación.
Estado de eventos globales	ReentrantReadWriteLock (fair)	Lectura concurrente, escritura atómica.
Pausa global remota (RMI)	ReentrantLock + Condition	Checkpoints en los hilos para pausa/reanudación.
Ranking RMI	CopyOnWriteArrayList<Demogorgon>	Muchas lecturas, pocas escrituras (spawns raros).

2.3. Flujo del Portal

El Portal es la pieza con mayor complejidad de sincronización. Combina formación de grupo exclusivo, cruce uno a uno, prioridad para los niños que retornan a Hawkins y bloqueo durante el apagón.

- Ida: 1) esperarGrupo() sobre la CyclicBarrier(groupSize). 2) cruceSem.acquire(). 3) esperarSiApagon(). 4) Espera a que retornando == 0. 5) Cruza 1 s. 6) zona.entrarNino(this). 7) Libera el semáforo.
- Vuelta: incrementa retornando, adquiere el semáforo y cruza inmediatamente; al salir, decrementa y señala puedeEntrar.signalAll() a las idas.
- Apagón: el flag portalBloqueado vive en EstadoGlobal; los que ya estan en el sleep finalizan, losnuevos esperan.

Portal: sincronizacion (grupo + cruce + prioridad + apagon)



Flujo de ida: 1) barrera 2) cruceSem.acquire() 3) esperarSiApagon 4) esperar retornando=0 5) cruzar 1s 6) zona.entrarNino 7) release

Figura 3. Sincronización del Portal.

2.4. Eventos globales aleatorios

GestorEventos es un hilo daemon que duerme entre 30 y 60 segundos y a continuación elige aleatoriamente uno de los cuatro eventos posibles (5–10 s de duracion). La activación se realiza bajo el writeLock de EstadoGlobal mediante los métodos lockEventoWrite() / unlockEventoWrite(); los flags se hacen visibles atómicamente y los hilos lectores los detectan en sus puntos de control.

- APAGON_LABORATORIO: portalBloqueado=true; los demogordons no cambian de zona pero siguen atacando.
- TORMENTA_UPSIDE_DOWN: multiplicadorSangre=2 y duración del ataque del demogorgón a la mitad.
- INTERVENCION_ELEVEN: paraliza demogordons y libera de la Colmena tantos niños como sangre haya recolectada (consumida vía consumirSangre()).
- RED_MENTAL: todos los demogorgons se dirigen a la zona con mas niños (zonaConMasNinos()).

2.5. Parte distribuida (RMI)

La interfaz remota HawkinsRemote expone dos métodos: getEstado() (devuelve un EstadoDTO serializable con un snapshot consistente) y setPausado(boolean). Elservidor (ServidorRemoto) extiende UnicastRemoteObject y publica el servicio en el registro RMI del puerto 1099.

El cliente (ClienteRemoto) refresca su GUI cada segundo con un `javax.swing.Timer` (sin intervención del usuario) y enlaza el botón rojo "PULSADOR DE EMERGENCIA" (un `JButton` transparente sobre la cúpula de la imagen) con `setPausado()`. La pausa real se implementa con un `ReentrantLock + Condition` en `EstadoGlobal`; los hilos invocan `checkPunto()` en puntos seguros y duermen hasta la reanudación.

3. Descripción de las clases principales

3.1. Paquete model

Nino extends Thread

Modela el ciclo de vida del niño. Atributos: id, estado, hawkins, portales, colmena, attackLock+attackFin (Condition), capturaLock+capturaCondition, portalElegido. Métodos clave: run(), ciclo(), explorarUpsideDown(), iniciarAtaque(idDemo), resolverAtaque(boolean), esperarFinAtaque(), esperarLiberacion(), liberarDeColmena(zona).

Demogorgon extends Thread

Patrulla aleatoriamente las zonas inseguras buscando niños. Atributos: id, colmena, estado, todasLasZonas, capturas (AtomicInteger), zonaActual, DEMO_COUNTER (estatico). Métodos clave: run(), atacar(Nino), moverseASiguienteZona(), elegirSiguienteZona(), esperarSiParalizado().

GestorEventos extends Thread (daemon)

Genera los eventos globales. Atributos: estado, colmena, zonas. Métodos clave: run(), iniciarEvento(t,duracion), terminarEvento(t), aplicarEfectosInicio(t), restaurarEfectos(t), zonaConMasNinos(). Usa lockEventoWrite()/unlockEventoWrite() de EstadoGlobal en lugar de exponer el writeLock directamente.

3.2. Paquete zones

ZonaHawkins

Tres listas (callePrincipal, sotanoByers, radioWsqq) protegidas por un único ReentrantLock(true). Expone snapshots inmutables que la GUI y el módulo RMI consumen sin bloquear las escrituras.

ZonaUpsideDown

Almacena los niños y demogorgons presentes en cada zona insegura. seleccionarNinoAleatorio() es atómica (comprueba y elige bajo el mismo lock) para evitar carreras niño/demo.

Portal

Combina CyclicBarrier(groupSize) para la formación del grupo, Semaphore(1, fair) para garantizar un único transeúnte y un par ReentrantLock + Condition para la prioridad del retorno. Las idas leen retornando.get() y se duermen en puedeEntrar.await() mientras haya retornos en curso.

Colmena

Lista capturados y contador totalCapturas. depositarNino() es synchronized: cada 8 capturas crea un nuevo demogordon y dispara el callback onSpawnDemogorgon (lo usa el ServidorRemoto para sumarlo al ranking sin acoplar paquetes).

3.3. Paquete state

EstadoGlobal es un singleton que centraliza el estado mutable: evento activo, flags por evento, sangre total y estado de pausa. Usa un ReentrantReadWriteLock(true) para que el gestor escriba en exclusión y los hilos consulten sin competir; el writeLock es PRIVADO y se accede vía los métodos públicos

lockEventoWrite()/unlockEventoWrite()). checkPunto() combina checkEvento() y checkPausa(). TipoEvento es el enum con los 4 valores (APAGON_LABORATORIO, TORMENTA_UPSIDE_DOWN, INTERVENCION_ELEVEN, RED_MENTAL).

3.4. Paquete log

SistemaLog: singleton thread-safe (Holder pattern). Escribe sobre hawkins.txt con un PrintWriter protegido por un ReentrantLock; añade un shutdown hook para cerrar el fichero.

3.5. Paquete remote

HawkinsRemote (intefaz Remote) declara los dos métodos getEstado() y setPausado(boolean). EstadoDTO es una clase Serializable con copias inmutables (Map.copyOf, List.copyOf) para garantizar la consistencia del snapshot. ServidorRemoto implementa HawkinsRemote y publica en localhost:1099; mantiene una CopyOnWriteArrayList de demogorgons activos para construir el ranking eficientemente. ClienteRemoto es la GUI Swing autónoma que se conecta al registro y refresca cada segundo.

3.6. Paquete gui

InicioGUI

Portada de arranque. Usa una imagen como fondo (inicio.png) y detecta el clic dentro del rectángulo del botón START mediante MouseListener. También responde a la tecla ENTER. Cuando se pulsa, ejecuta el Runnable onStart pasado por Main, que arranca el sistema completo (zonas, portales, hilos, RMI, GUIs).

HawkinsGUI

Ventana principal del servidor con la imagen hawkins_server.png como fondo (IbIBg). 23 JLabels superpuestos en posiciones absolutas (AbsoluteLayout) muestran los datos dinámicos: niños en cada zona de Hawkins, niños esperando en cada portal, niños/demogorgons en cada zona del Upside Down, contador de la Colmena, sangre acumulada y evento global activo. Refresco cada 500 ms con javax.swing.Timer.

ClienteRemoto

GUI del módulo RMI con la imagen consola.png como fondo. JLabels absolutos para sujetos en Hawkins, niños por portal, niños por zona en SUPERFICIE y OTRO LADO, contador Colmena, ranking top-3 y descripción del evento activo + temporizador. JButton transparente sobre la cúpula roja del PULSADOR DE EMERGENCIA: alterna setPausado() en el servidor.

4. Diagrama de clases

El siguiente diagrama resume las clases principales con sus atributos y métodos más relevantes y las relaciones de asociación entre ellas. Las clases estereotipadas como «Thread» son los hilos del sistema; las marcadas como «Singleton» son únicas (acceso global controlado); «enum» indica una enumeración.

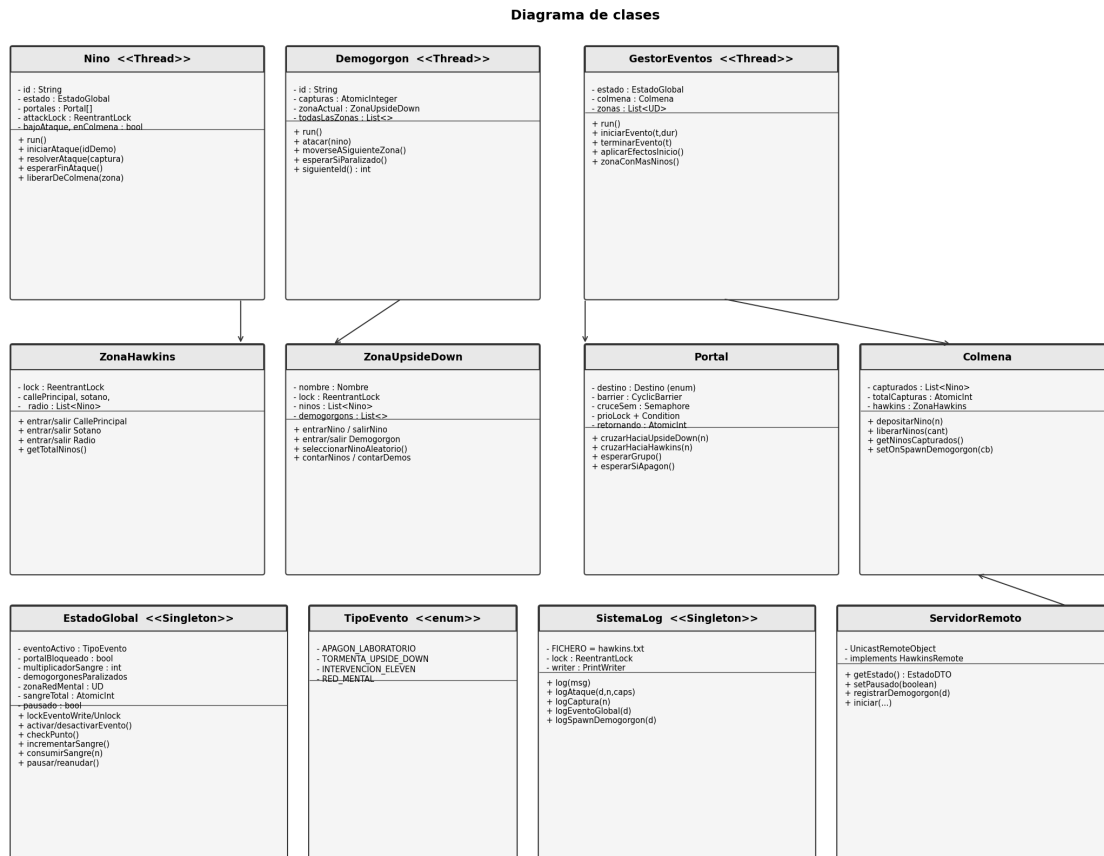


Figura 4. Diagrama de clases del sistema.

Diagrama de secuencia: ataque y captura

Secuencia: ataque y captura

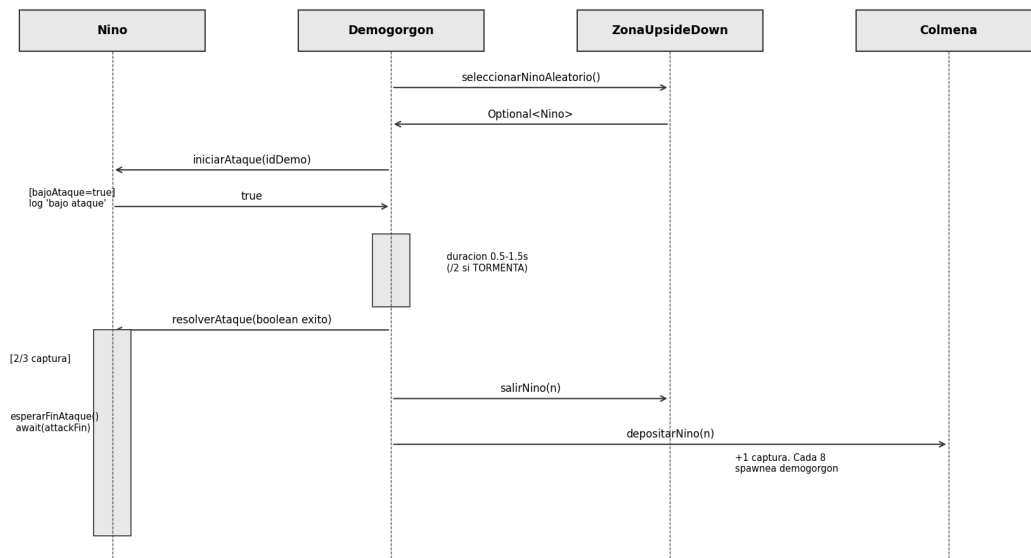


Figura 5. Secuencia: ataque y captura de un niño.

5. Capturas de la interfaz

La aplicación se compone de tres ventanas Swing. Al ejecutar Main aparece primero la portada (InicioGUI). Al pulsar el botón INICIAR (START) o la tecla ENTER, se cierra la portada y se lanzan simultáneamente la GUI principal del servidor (HawkinsGUI) y la GUI del módulo remoto (ClienteRemoto). Las tres ventanas usan AbsoluteLayout con una imagen de fondo y JLabels superpuestos en posiciones absolutas.

5.1. Portada (InicioGUI)



Figura 6. Portada de arranque del sistema.

5.2. Servidor (HawkinsGUI)

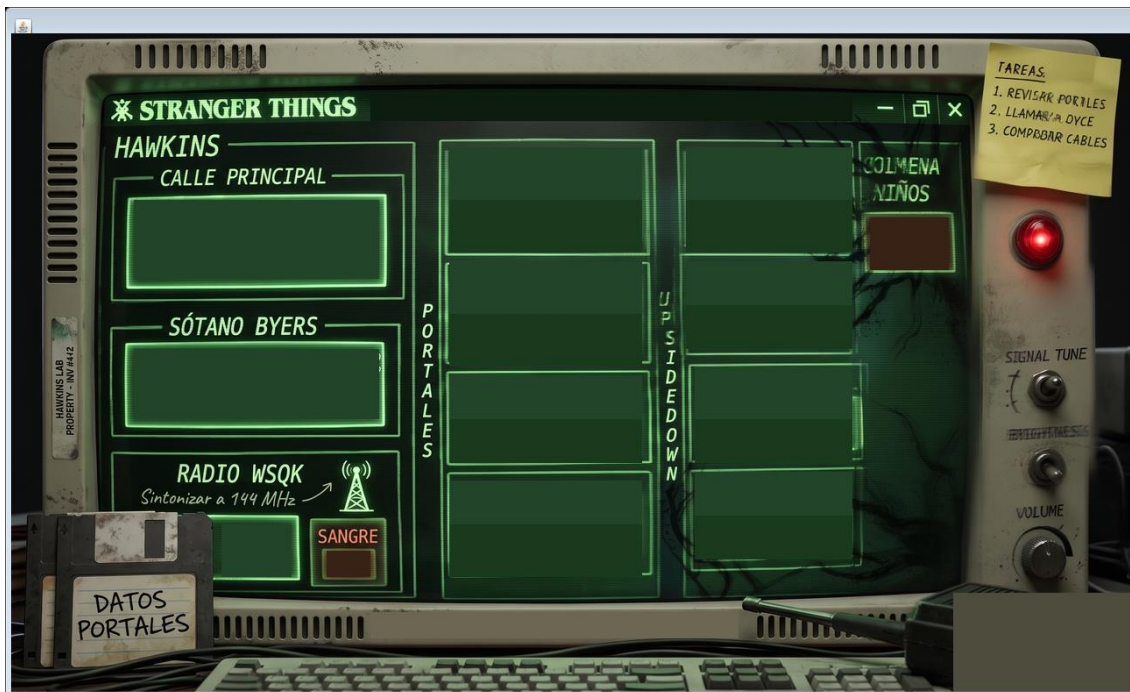


Figura 7. HawkinsGUI — terminal del operador con datos sintéticos.

5.3. Cliente RMI (ClienteRemoto)



Figura 8. ClienteRemoto — consola de monitoreo con la cúpula roja del pulsador de emergencia.

6. Anexo: código fuente

Se incluye a continuación el código fuente completo del proyecto en el orden lógico de lectura. Todas las clases han sido desarrolladas específicamente para esta práctica.

// Main.java

```
import gui.InicioGUI;
import log.SistemaLog;
import model.Demogorgon;
import model.GestorEventos;
import model.Nino;
import state.EstadoGlobal;
import zones.Colmena;
import zones.Portal;
import zones.ZonaHawkins;
import zones.ZonaUpsideDown;

import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Random;

public class Main {

    private static final int    TOTAL_NINOS = 1500;
    private static final Random RNG         = new Random();

    public static void main(String[] args) {
        // Forzar UTF-8 para que tildes y ñ salgan bien en consola
        System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8));
        System.setErr(new PrintStream(System.err, true, StandardCharsets.UTF_8));

        javax.swing.SwingUtilities.invokeLater(() ->
            new InicioGUI(Main::arrancarSistema).setVisible(true));
    }

    private static void arrancarSistema() {
        try {
            SistemaLog.getInstance().log("=== Sistema Hawkins iniciado ===");

            ZonaUpsideDown bosque      = new ZonaUpsideDown(ZonaUpsideDown.Nombre.BOSQUE);
            ZonaUpsideDown laboratorio = new ZonaUpsideDown(ZonaUpsideDown.Nombre.LABORATORIO);
            ZonaUpsideDown centro      = new ZonaUpsideDown(ZonaUpsideDown.Nombre.CENTRO_COMERCIAL);
            ZonaUpsideDown alcant      = new ZonaUpsideDown(ZonaUpsideDown.Nombre.ALCANTARILLADO);
            List<ZonaUpsideDown> todasLasZonas = List.of(bosque, laboratorio, centro, alcant);

            ZonaHawkins hawkins = new ZonaHawkins();
            Colmena      colmena = new Colmena(hawkins);

            Portal[] portales = {
                new Portal(Portal.Destino.BOSQUE,      bosque),
                new Portal(Portal.Destino.LABORATORIO, laboratorio),
                new Portal(Portal.Destino.CENTRO_COMERCIAL, centro),
                new Portal(Portal.Destino.ALCANTARILLADO, alcant)
            };

            EstadoGlobal.setZonasUpsideDown(todasLasZonas);
            EstadoGlobal estado = EstadoGlobal.getInstance();

            javax.swing.SwingUtilities.invokeLater(() ->
                new gui.HawkinsGUI(hawkins, portales, todasLasZonas, colmena, estado)
                    .setVisible(true));

            // Si el RMI falla sigo sin él, no se cae todo
            try {
                remote.ServidorRemoto.iniciar(hawkins, portales, todasLasZonas, colmena, estado);
                colmena.setOnSpawnDemogorgon(remote.ServidorRemoto::registrarDemogorgon);
            } catch (Exception e) {
```

```

        SistemaLog.getInstance().log(
            "WARN: Servidor RMI no iniciado: " + e.getMessage());
    }

    javax.swing.SwingUtilities.invokeLater(() ->
        new remote.ClienteRemoto("localhost").setVisible(true));

    new GestorEventos(estado, colmena, todasLasZonas).start();

    // Demogorgon 0
    Demogorgon alpha = new Demogorgon("D0000", colmena, estado, todasLasZonas);
    alpha.setDaemon(false);
    remote.ServidorRemoto.registrarDemogorgon(alpha);
    alpha.start();
    SistemaLog.getInstance().log("Demogorgon Alpha D0000 inicia patrullaje");

    // Spawneo escalonado
    Thread spawner = new Thread(() -> {
        SistemaLog.getInstance().log(
            "Iniciando creación escalonada de " + TOTAL_NINOS + " niños...");
        for (int i = 1; i <= TOTAL_NINOS; i++) {
            String id = String.format("N%04d", i);
            Nino nino = new Nino(id, hawkins, portales, colmena, estado);
            nino.setDaemon(false);
            nino.start();
            try { Thread.sleep(500 + (long)(RNG.nextDouble() * 1500)); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); break; }
        }
        SistemaLog.getInstance().log(
            "Todos los " + TOTAL_NINOS + " niños creados. Sistema en marcha.");
    }, "Hilo-SpawnerNinos");
    spawner.setDaemon(false);
    spawner.start();
} catch (Exception ex) {
    ex.printStackTrace();
}
}
}

```

// TipoEvento.java

```

package state;

public enum TipoEvento {
    APAGON_LABORATORIO,
    TORMENTA_UPSIDE_DOWN,
    INTERVENCION_ELEVEN,
    RED_MENTAL
}

```

// EstadoGlobal.java

```

package state;

import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public final class EstadoGlobal {

    private static final EstadoGlobal INSTANCE = new EstadoGlobal();
    private EstadoGlobal() {}
    public static EstadoGlobal getInstance() { return INSTANCE; }

    // RW-Lock: muchos hilos leen el estado del evento, solo el gestor escrib
    private final ReentrantReadWriteLock eventoRWLock =
        new ReentrantReadWriteLock(true);
    private final ReentrantReadWriteLock writeLock =
        eventoRWLock.writeLock();
}

```

```

private final ReentrantReadWriteLock.ReadLock readLock =
    eventoRWLock.readLock();

public void lockEventoWrite() { writeLock.lock(); }
public void unlockEventoWrite() { writeLock.unlock(); }

public void checkEvento() throws InterruptedException {
    readLock.lockInterruptibly();
    readLock.unlock();
}

private volatile TipoEvento eventoActivo = null;
private volatile long eventoInicioMs = 0;
private volatile long eventoDuracionMs = 0;

public void activarEvento(TipoEvento tipo, long duracionMs) {
    this.eventoActivo = tipo;
    this.eventoInicioMs = System.currentTimeMillis();
    this.eventoDuracionMs = duracionMs;
}

public void desactivarEvento() { this.eventoActivo = null; }

public TipoEvento getEventoActivo() { return eventoActivo; }

public long getTiempoRestanteMs() {
    if (eventoActivo == null) return 0L;
    return Math.max(0L,
        eventoDuracionMs - (System.currentTimeMillis() - eventoInicioMs));
}

// Volatile porque los hilos los miran en cada iteración sin lock
private volatile boolean portalBloqueado = false;
private volatile int multiplicadorSangre = 1;
private volatile boolean demogorgonesParalizados = false;
private volatile zones.ZonaUpsideDown zonaRedMental = null;

public boolean isPortalBloqueado() { return portalBloqueado; }
public int getMultiplicadorSangre() { return multiplicadorSangre; }
public boolean isDemogorgonesParalizados() { return demogorgonesParalizados; }
public zones.ZonaUpsideDown getZonaRedMental() { return zonaRedMental; }

public void setPortalBloqueado(boolean v) { portalBloqueado = v; }
public void setMultiplicadorSangre(int v) { multiplicadorSangre = v; }
public void setDemogorgonesParalizados(boolean v) { demogorgonesParalizados = v; }
public void setZonaRedMental(zones.ZonaUpsideDown z) { zonaRedMental = z; }

// Pausa pedida desde RMI
private final ReentrantLock pauseLock = new ReentrantLock();
private final Condition pauseCondition = pauseLock.newCondition();
private volatile boolean pausado = false;

public void pausar() {
    pauseLock.lock();
    try { pausado = true; }
    finally { pauseLock.unlock(); }
}

public void reanudar() {
    pauseLock.lock();
    try { pausado = false; pauseCondition.signalAll(); }
    finally { pauseLock.unlock(); }
}

public void checkPausa() throws InterruptedException {
    if (!pausado) return;
    pauseLock.lock();
    try { while (pausado) pauseCondition.await(); }
    finally { pauseLock.unlock(); }
}

// Lo llaman los hilos en sitios seguros para chequear evento y pausa

```

```

    public void checkPunto() throws InterruptedException {
        checkEvento();
        checkPausa();
    }

    public boolean isPausado() { return pausado; }

    private final AtomicInteger sangreTotal = new AtomicInteger(0);

    public int incrementarSangre() { return sangreTotal.incrementAndGet(); }
    public int getSangreTotal() { return sangreTotal.get(); }

    // CAS para que dos hilos no resten a la vez y se pase por debajo de 0
    public int consumirSangre(int n) {
        int actual, nuevo;
        do {
            actual = sangreTotal.get();
            nuevo = Math.max(0, actual - n);
        } while (!sangreTotal.compareAndSet(actual, nuevo));
        return actual - nuevo;
    }

    private static volatile List<zones.ZonaUpsideDown> zonasUpsideDown;

    public static void setZonasUpsideDown(List<zones.ZonaUpsideDown> z) {
        zonasUpsideDown = z;
    }
    public static List<zones.ZonaUpsideDown> getZonasUpsideDown() {
        return zonasUpsideDown;
    }
}

```

// SistemaLog.java

```

package log;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.locks.ReentrantLock;

public final class SistemaLog {

    private static final String FICHERO = "hawkins.txt";
    private static final DateTimeFormatter FORMATO =
        DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

    // Holder pattern: instancia perezosa y thread-safe sin synchronized
    private static final class Holder {
        static final SistemaLog INSTANCE = new SistemaLog();
    }

    private final ReentrantLock lock = new ReentrantLock();
    private PrintWriter writer;

    private SistemaLog() {
        try {
            writer = new PrintWriter(
                new BufferedWriter(new FileWriter(FICHERO, true)), false);
        } catch (IOException e) {
            throw new RuntimeException("No se puede abrir: " + FICHERO, e);
        }
        Runtime.getRuntime().addShutdownHook(new Thread(this::cerrar));
    }

    public static SistemaLog getInstance() {
        return Holder.INSTANCE;
    }
}

```



```

    }

    // Lock para que dos hilos no escriban líneas mezcladas
    public void log(String mensaje) {
        String linea = "[" + LocalDateTime.now().format(FORMATO) + "]" + mensaje;
        System.out.println(linea);
        lock.lock();
        try {
            if (writer != null) {
                writer.println(linea);
                writer.flush();
            }
        } finally {
            lock.unlock();
        }
    }

    public void logAtaque(String idDemo, String idNino, int capturas) {
        log("El demogorgon " + idDemo + " ataca al niño " + idNino +
            " (capturas: " + capturas + ")");
    }

    public void logCaptura(String idNino) {
        log("El niño " + idNino + " ha sido capturado");
    }

    public void logEventoGlobal(String descripcion) {
        log("EVENTO GLOBAL: " + descripcion);
    }

    public void logSpawnDemogorgon(String idDemo) {
        log("Vecna genera un nuevo demogorgon: " + idDemo);
    }

    public void logLibreradoPorEleven(String idNino) {
        log("Eleven libera al niño " + idNino + " de la Colmena");
    }

    private void cerrar() {
        lock.lock();
        try {
            if (writer != null) {
                writer.flush();
                writer.close();
                writer = null;
            }
        } finally {
            lock.unlock();
        }
    }
}

```

// ZonaHawkins.java

```

package zones;

import model.Nino;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

public class ZonaHawkins {

    // Un solo lock para las 3 litas: simplifica las capturas para la GUI
    private final ReentrantLock lock = new ReentrantLock(true);

    private final List<Nino> callePrincipal = new ArrayList<>();
    private final List<Nino> sotanoByers = new ArrayList<>();
}

```

```

private final List<Nino> radioWsqs = new ArrayList<>();

public void entrarCallePrincipal(Nino n) {
    lock.lock();
    try { callePrincipal.add(n); }
    finally { lock.unlock(); }
}

public void salirCallePrincipal(Nino n) {
    lock.lock();
    try { callePrincipal.remove(n); }
    finally { lock.unlock(); }
}

public void entrarSotano(Nino n) {
    lock.lock();
    try { sotanoByers.add(n); }
    finally { lock.unlock(); }
}

public void salirSotano(Nino n) {
    lock.lock();
    try { sotanoByers.remove(n); }
    finally { lock.unlock(); }
}

public void entrarRadio(Nino n) {
    lock.lock();
    try { radioWsqs.add(n); }
    finally { lock.unlock(); }
}

public void salirRadio(Nino n) {
    lock.lock();
    try { radioWsqs.remove(n); }
    finally { lock.unlock(); }
}

public int getTotalNinos() {
    lock.lock();
    try {
        return callePrincipal.size() + sotanoByers.size() + radioWsqs.size();
    } finally { lock.unlock(); }
}

public List<String> getCallePrincipalIds() { return snapshot(callePrincipal); }
public List<String> getSotanoIds() { return snapshot(sotanoByers); }
public List<String> getRadioIds() { return snapshot(radioWsqs); }

// Copia la GUI puede iterar sin miedo a modificaciones a la vez
private List<String> snapshot(List<Nino> lista) {
    lock.lock();
    try {
        List<String> ids = new ArrayList<>(lista.size());
        lista.forEach(n -> ids.add(n.getNombre()));
        return Collections.unmodifiableList(ids);
    } finally { lock.unlock(); }
}
}

```

// ZonaUpsideDown.java

```

package zones;

import model.Nino;
import model.Demogorgon;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

```

```

import java.util.Optional;
import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

public class ZonaUpsideDown {

    public enum Nombre {
        BOSQUE, LABORATORIO, CENTRO_COMERCIAL, ALCANTARILLADO
    }

    private final Nombre nombre;
    private final ReentrantLock lock = new ReentrantLock(true);
    private final List<Nino> ninos = new ArrayList<>();
    private final List<Demogorgon> demogorgons = new ArrayList<>();
    private final Random rng = new Random();

    public ZonaUpsideDown(Nombre nombre) {
        this.nombre = nombre;
    }

    public void entrarNino(Nino nino) {
        lock.lock();
        try { ninos.add(nino); }
        finally { lock.unlock(); }
    }

    public void salirNino(Nino nino) {
        lock.lock();
        try { ninos.remove(nino); }
        finally { lock.unlock(); }
    }

    // Comprobar y elegir el mismo lock para evitar carreras niño/demo
    public Optional<Nino> seleccionarNinoAleatorio() {
        lock.lock();
        try {
            if (ninos.isEmpty()) return Optional.empty();
            return Optional.of(ninos.get(rng.nextInt(ninos.size())));
        } finally {
            lock.unlock();
        }
    }

    public void entrarDemogorgon(Demogorgon d) {
        lock.lock();
        try { demogorgons.add(d); }
        finally { lock.unlock(); }
    }

    public void salirDemogorgon(Demogorgon d) {
        lock.lock();
        try { demogorgons.remove(d); }
        finally { lock.unlock(); }
    }

    public int contarNinos() {
        lock.lock();
        try { return ninos.size(); }
        finally { lock.unlock(); }
    }

    public int contarDemogorgons() {
        lock.lock();
        try { return demogorgons.size(); }
        finally { lock.unlock(); }
    }

    public List<String> getNinoIds() {
        lock.lock();
        try {
            List<String> ids = new ArrayList<>(ninos.size());
            ninos.forEach(n -> ids.add(n.getNombre()));
        }
    }
}

```

```

        return Collections.unmodifiableList(ids);
    } finally { lock.unlock(); }
}

public List<String> getDemogorgonIds() {
    lock.lock();
    try {
        List<String> ids = new ArrayList<>(demogorgons.size());
        demogorgons.forEach(d -> ids.add(d.getNombre()));
        return Collections.unmodifiableList(ids);
    } finally { lock.unlock(); }
}

public Nombre getNombre() { return nombre; }

@Override
public String toString() { return nombre.name(); }
}

```

// Portal.java

```

package zones;

import model.Nino;
import state.EstadoGlobal;
import log.SistemaLog;

import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class Portal {

    public enum Destino {
        BOSQUE(2), LABORATORIO(3), CENTRO_COMERCIAL(4), ALCANTARILLADO(2);

        public final int groupSize;
        Destino(int g) { this.groupSize = g; }
    }

    private final Destino destino;
    private final ZonaUpsideDown zona;
    private final EstadoGlobal estado = EstadoGlobal.getInstance();

    // CyclicBarrier para juntar el grupo del tamaño exacto que pide cada portal
    private CyclicBarrier barrier;
    private final ReentrantLock barrierResetLock = new ReentrantLock();
    // Semáforo para que solo cruce 1 niño cada vez (en cualquier sentido)
    private final Semaphore cruceSem = new Semaphore(1, true);
    // Lock + condition para dar prioridad a los retornos sobre las idas
    private final ReentrantLock prioLock = new ReentrantLock(true);
    private final Condition puedeEntrar = prioLock.newCondition();
    private final AtomicInteger retornando = new AtomicInteger(0);
    private final AtomicInteger esperandoEntrada = new AtomicInteger(0);
    private final AtomicInteger esperandoSalida = new AtomicInteger(0);

    public Portal(Destino destino, ZonaUpsideDown zona) {
        this.destino = destino;
        this.zona = zona;
        this.barrier = new CyclicBarrier(destino.groupSize);
    }

    public void cruzarHaciaUpsideDown(Nino nino) throws InterruptedException {
        esperandoEntrada.incrementAndGet();
        try {
            esperarGrupo(nino);

```

```

    cruceSem.acquire();
    try {
        esperarSiApagon(nino);

        // Si hay alguien volviendo, cedo el turno
        prioLock.lock();
        try {
            while (retornando.get() > 0) puedeEntrar.await();
        } finally {
            prioLock.unlock();
        }

        SistemaLog.getInstance().log(
            nino.getNombre() + " cruza portal " + destino + " → Upside Down");
        Thread.sleep(1000);
        zona.entrarNino(nino);
    } finally {
        cruceSem.release();
    }
} finally {
    esperandoEntrada.decrementAndGet();
}
}

private void esperarGrupo(Nino nino) throws InterruptedException {
    while (true) {
        esperarSiApagon(nino);
        try {
            barrier.await();
            return;
        } catch (BrokenBarrierException e) {
            // Si la barrera se rompe, la reseteo y vuelvo a intentar
            if (barrierResetLock.tryLock()) {
                try {
                    if (barrier.isBroken()) barrier.reset();
                } finally {
                    barrierResetLock.unlock();
                }
            }
            Thread.sleep(50);
        }
    }
}

private void esperarSiApagon(Nino nino) throws InterruptedException {
    while (estado.isPortalBloqueado()) {
        SistemaLog.getInstance().log(
            nino.getNombre() + " espera en portal " + destino + " (apagón)");
        Thread.sleep(300);
    }
}

public void cruzarHaciaHawkins(Nino nino) throws InterruptedException {
    esperandoSalida.incrementAndGet();
    retornando.incrementAndGet();
    try {
        cruceSem.acquire();
        try {
            zona.salirNino(nino);
            SistemaLog.getInstance().log(
                nino.getNombre() + " cruza portal " + destino + " → Hawkins");
            Thread.sleep(1000);
        } finally {
            cruceSem.release();
        }
    } finally {
        esperandoSalida.decrementAndGet();
        // Cuando ya no quedan retorno, despierto a las idas
        if (retornando.decrementAndGet() == 0) {
            prioLock.lock();
            try { puedeEntrar.signalAll(); }
            finally { prioLock.unlock(); }
        }
    }
}

```

```

    }
}

public Destino      getDestino()      { return destino; }
public ZonaUpsideDown getZona()      { return zona; }
public int          getEsperandoEntrada() { return esperandoEntrada.get(); }
public int          getEsperandoSalida() { return esperandoSalida.get(); }
public int          getTotalEsperando() {
    return esperandoEntrada.get() + esperandoSalida.get();
}
}
}

```

// Colmena.java

```

package zones;

import model.Nino;
import model.Demogorgon;
import state.EstadoGlobal;
import log.SistemaLog;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Consumer;

public class Colmena {

    private static final int CAPTURAS_POR_SPAWN = 8;

    private final List<Nino>   capturados   = new ArrayList<>();
    private final AtomicInteger totalCapturas = new AtomicInteger(0);
    private final ZonaHawkins hawkins;

    // Callback lo usa el ServidorRemoto para meter el nuevo demo en el ranking
    private Consumer<Demogorgon> onSpawnDemogorgon = d -> {};

    public Colmena(ZonaHawkins hawkins) {
        this.hawkins = hawkins;
    }

    public void setOnSpawnDemogorgon(Consumer<Demogorgon> callback) {
        this.onSpawnDemogorgon = callback;
    }

    public synchronized void depositarNino(Nino nino) {
        capturados.add(nino);
        SistemaLog.getInstance().logCaptura(nino.getNombre());

        // Cada 8 capturas Vecna escupe otro demogorgon
        if (totalCapturas.incrementAndGet() % CAPTURAS_POR_SPAWN == 0) {
            String idNuevo = String.format("D%04d", Demogorgon.siguienteId());
            SistemaLog.getInstance().logSpawnDemogorgon(idNuevo);

            Demogorgon nuevo = new Demogorgon(
                idNuevo, this, EstadoGlobal.getInstance());
            nuevo.setDaemon(false);
            onSpawnDemogorgon.accept(nuevo);
            nuevo.start();
        }
    }

    // Lo llama Eleven en su evento: libera 'cantidad' niños y los manda a Calle Principal
    public synchronized int liberarNinos(int cantidad) {
        int liberados = Math.min(cantidad, capturados.size());
        for (int i = liberados - 1; i >= 0; i--) {
            Nino nino = capturados.remove(i);
            SistemaLog.getInstance().logLibreradoPorEleven(nino.getNombre());
        }
    }
}

```

```

        nino.liberarDeColmena(hawkins);
    }
    return liberados;
}

public int getTotalCapturas() { return totalCapturas.get(); }

public synchronized int getNinosCapturados() { return capturados.size(); }

public synchronized List<String> getCapturadosIds() {
    List<String> ids = new ArrayList<>(capturados.size());
    capturados.forEach(n -> ids.add(n.getNombre()));
    return Collections.unmodifiableList(ids);
}
}

```

// Nino.java

```

package model;

import log.SistemaLog;
import state.EstadoGlobal;
import zones.Colmena;
import zones.Portal;
import zones.ZonaHawkins;

import java.util.Random;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class Nino extends Thread {

    private static final Random RNG = new Random();

    private final String id;
    private final EstadoGlobal estado;
    private final ZonaHawkins hawkins;
    private final Portal[] portales;

    // El demo escribe estos flags y avisa, el niño solo espera (al revés sería deadlock)
    private final ReentrantLock attackLock = new ReentrantLock();
    private final Condition attackFin = attackLock.newCondition();
    private volatile boolean bajoAtaque = false;

    // Para dormir al niño mientras está en la Colmena
    private final ReentrantLock capturaLock = new ReentrantLock();
    private final Condition capturaCondition = capturaLock.newCondition();
    private volatile boolean enColmena = false;

    private Portal portalElegido;

    public Nino(String id, ZonaHawkins hawkins, Portal[] portales,
                Colmena colmena, EstadoGlobal estado) {
        super("Hilo-" + id);
        this.id = id;
        this.hawkins = hawkins;
        this.portales = portales;
        this.estado = estado;
    }

    @Override
    public void run() {
        try {
            hawkins.entrarCallePrincipal(this);
            SistemaLog.getInstance().log(
                id + " inicia en Calle Principal de Hawkins");

            // El enunciado pide que el ciclo no acabe nunca
            while (!Thread.currentThread().isInterrupted()) {
                estado.checkPunto();
            }
        }
    }
}

```

```

        ciclo();
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}

private void ciclo() throws InterruptedException {

    hawkins.salirCallePrincipal(this);
    hawkins.entrarSotano(this);
    SistemaLog.getInstance().log(id + " llega al Sótano Byers, preparándose");
    dormir(1000, 2000);
    estado.checkPunto();

    portalElegido = portales[RNG.nextInt(portales.length)];
    SistemaLog.getInstance().log(
        id + " elige portal " + portalElegido.getDestino() + " y espera grupo");
    hawkins.salirSotano(this);
    portalElegido.cruzarHaciaUpsideDown(this);
    estado.checkPunto();

    if (!enColmena) {
        explorarUpsideDown();
        // Si el demo me atacó al final de la exploración, espero a que resuelva
        esperarFinAtaque();
    }

    if (enColmena) {
        SistemaLog.getInstance().log(
            id + " está capturado en la Colmena, esperando a Eleven");
        esperarLiberacion();
        return;
    }

    int total = estado.incrementarSangre();
    SistemaLog.getInstance().log(
        id + " recoge sangre de Vecna (total acumulado: " + total + ")");
    portalElegido.cruzarHaciaHawkins(this);
    estado.checkPunto();

    hawkins.entrarRadio(this);
    SistemaLog.getInstance().log(id + " llega a Radio WSQK");
    dormir(2000, 4000);
    hawkins.salirRadio(this);
    estado.checkPunto();

    hawkins.entrarCallePrincipal(this);
    SistemaLog.getInstance().log(id + " deambula por Calle Principal");
    dormir(3000, 5000);
}

private void explorarUpsideDown() throws InterruptedException {
    // Multiplicador 1 normal, 2 si hay TORMENTA
    long base = 3000L + (long)(RNG.nextDouble() * 2000L);
    long ms = base * estado.getMultiplicadorSangre();
    SistemaLog.getInstance().log(
        id + " explora el Upside Down (" + ms / 1000 + "s"
        + (estado.getMultiplicadorSangre() > 1 ? ", TORMENTA activa" : "") + ")");
    Thread.sleep(ms);
}

// Lo llama el demo. Devuelve false si ya hay un ataque o ya está capturado
public boolean iniciarAtaque(String idDemo) {
    attackLock.lock();
    try {
        if (bajoAtaque || enColmena) return false;
        bajoAtaque = true;
        SistemaLog.getInstance().log(id + " bajo ataque de " + idDemo);
        return true;
    } finally {
        attackLock.unlock();
    }
}

```



```

    }
}

// Lo llama el demo al acabar el ataque, con el resultado
public void resolverAtaque(boolean capturar) {
    attackLock.lock();
    try {
        // Pongo enColmena ANTES de signal para que el niño vea el valor correcto al despierta
        if (capturar) enColmena = true;
        bajoAtaque = false;
        attackFin.signalAll();
    } finally {
        attackLock.unlock();
    }
}

private void esperarFinAtaque() throws InterruptedException {
    attackLock.lock();
    try {
        while (bajoAtaque) attackFin.await();
    } finally {
        attackLock.unlock();
    }
}

private void esperarLiberacion() throws InterruptedException {
    capturaLock.lock();
    try {
        while (enColmena) capturaCondition.await();
    } finally {
        capturaLock.unlock();
    }
}

// Lo llama Colmena.liberarNinos() desde GestorEventos
public void liberarDeColmena(ZonaHawkins destino) {
    capturaLock.lock();
    try {
        enColmena = false;
        destino.entrarCallePrincipal(this);
        capturaCondition.signalAll();
    } finally {
        capturaLock.unlock();
    }
}

private void dormir(long minMs, long maxMs) throws InterruptedException {
    Thread.sleep(minMs + (long)(RNG.nextDouble() * (maxMs - minMs)));
}

public String getNombre() { return id; }
public boolean isEnColmena() { return enColmena; }
public boolean isBajoAtaque() { return bajoAtaque; }
}

```

// Demogorgon.java

```

package model;

import log.SistemaLog;
import state.EstadoGlobal;
import state.TipoEvento;
import zones.Colmena;
import zones.ZonaUpsideDown;

import java.util.List;
import java.util.Optional;
import java.util.Random;
import java.util.concurrent.atomic.AtomicInteger;

```

```

public class Demogorgon extends Thread {

    private static final Random RNG = new Random();

    // Contador global de IDs. Solo se muve cuando Colmena spawna uno nuevo
    private static final AtomicInteger DEMO_COUNTER = new AtomicInteger(1);

    private final String            id;
    private final Colmena           colmena;
    private final EstadoGlobal      estado;
    private final List<ZonaUpsideDown> todasLasZonas;

    // Capturas de ESTE demo, las lee el ranking RMI sin parar
    private final AtomicInteger capturas = new AtomicInteger(0);

    private ZonaUpsideDown zonaActual;

    public Demogorgon(String id, Colmena colmena, EstadoGlobal estado,
        List<ZonaUpsideDown> zonas) {
        super("Hilo-" + id);
        this.id = id;
        this.colmena = colmena;
        this.estado = estado;
        this.todasLasZonas = zonas;
        // Empieza en una zona aleatoria
        this.zonaActual = zonas.get(RNG.nextInt(zonas.size()));
    }

    public Demogorgon(String id, Colmena colmena, EstadoGlobal estado) {
        this(id, colmena, estado, EstadoGlobal.getZonasUpsideDown());
    }

    public static int siguienteId() {
        return DEMO_COUNTER.getAndIncrement();
    }

    @Override
    public void run() {
        zonaActual.entrarDemogorgon(this);
        SistemaLog.getInstance().log(
            id + " inicia patrullaje en " + zonaActual);

        try {
            while (!Thread.currentThread().isInterrupted()) {
                estado.checkPunto();
                esperarSiParalizado();

                Optional<Nino> objetivo = zonaActual.seleccionarNinoAleatorio();

                if (objetivo.isPresent()) {
                    atacar(objetivo.get());
                } else {
                    SistemaLog.getInstance().log(
                        id + " no encuentra niños en " + zonaActual + ", esperando");
                    dormir(4000, 5000);
                }

                estado.checkPunto();
                esperarSiParalizado();

                moverseASiguienteZona();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void atacar(Nino nino) throws InterruptedException {
        boolean iniciado = nino.iniciarAtaque(id);
        if (!iniciado) {
            // Otro demo ya lo está atacando: pequeño descanso para no quemar CPU
            Thread.sleep(200);
        }
    }
}

```

```

        return;
    }

    // Duración base 0.5-1.5s, mitad si TORMENTA
    double factor = (estado.getEventoActivo() == TipoEvento.TORMENTA_UPSIDE_DOWN)
        ? 0.5 : 1.0;
    long duracion = (long)((500 + RNG.nextInt(1000)) * factor);

    Thread.sleep(duracion);

    // 2/3 de éxito
    boolean exito = RNG.nextInt(3) < 2;
    nino.resolverAtaque(exito);

    if (exito) {
        int totalCapturas = capturas.incrementAndGet();
        SistemaLog.getInstance().logAtaque(id, nino.getNombre(), totalCapturas);

        // Tiempo de depósito en la Colmena
        Thread.sleep(500 + RNG.nextInt(500));

        zonaActual.salirNino(nino);
        colmena.depositarNino(nino);
    } else {
        SistemaLog.getInstance().log(
            id + " ataca a " + nino.getNombre() +
            " pero el niño resiste (probabilidad 1/3)");
    }
}

private void moverseASiguienteZona() throws InterruptedException {
    // Durante el apagón los demos no cambian de zona
    if (estado.isPortalBloqueado()) {
        SistemaLog.getInstance().log(
            id + " permanece en " + zonaActual + " (apagón activo)");
        return;
    }
    ZonaUpsideDown destino = elegirSiguieteZona();

    if (destino == zonaActual) return;

    zonaActual.salirDemogorgon(this);
    zonaActual = destino;
    zonaActual.entrarDemogorgon(this);
    SistemaLog.getInstance().log(id + " se desplaza a " + zonaActual);
}

private ZonaUpsideDown elegirSiguieteZona() {
    // RED MENTAL: todos van a la zona con más niños
    ZonaUpsideDown redMental = estado.getZonaRedMental();
    if (redMental != null) {
        SistemaLog.getInstance().log(
            id + " recibe orden de Red Mental: ir a " + redMental);
        return redMental;
    }

    // Modo normal: cualquier zona menos la actual
    List<ZonaUpsideDown> otras = todasLasZonas.stream()
        .filter(z -> z != zonaActual)
        .toList();
    return otras.get(RNG.nextInt(otras.size()));
}

private void esperarSiParalizado() throws InterruptedException {
    if (!estado.isDemogorgonesParalizados()) return;
    SistemaLog.getInstance().log(id + " paralizado por Eleven");
    while (estado.isDemogorgonesParalizados()) {
        Thread.sleep(100);
    }
    SistemaLog.getInstance().log(id + " reanuda actividad tras Eleven");
}
}

```

```

        private void dormir(long minMs, long maxMs) throws InterruptedException {
            Thread.sleep(minMs + (long)(RNG.nextDouble() * (maxMs - minMs)));
        }

        public String getNombre() { return id; }
        public int getCapturas() { return capturas.get(); }

        @Override
        public String toString() { return id; }
    }

```

// GestorEventos.java

```

package model;

import log.SistemaLog;
import state.EstadoGlobal;
import state.TipoEvento;
import zones.Colmena;
import zones.ZonaUpsideDown;

import java.util.List;
import java.util.Random;

public class GestorEventos extends Thread {

    private static final Random RNG = new Random();

    private final EstadoGlobal estado;
    private final Colmena colmena;
    private final List<ZonaUpsideDown> zonas;

    public GestorEventos(EstadoGlobal estado, Colmena colmena,
                        List<ZonaUpsideDown> zonas) {
        super("Hilo-GestorEventos");
        this.estado = estado;
        this.colmena = colmena;
        this.zonas = zonas;
        // Daemon: muere solo cuando los hilos no-daemon terminen
        setDaemon(true);
    }

    @Override
    public void run() {
        try {
            while (!Thread.currentThread().isInterrupted()) {

                // Espera 30-60s entre eventos
                Thread.sleep(30_000 + (long)(RNG.nextDouble() * 30_000));

                TipoEvento tipo = elegirEvento();
                long duracion = 5_000 + (long)(RNG.nextDouble() * 5_000);

                iniciarEvento(tipo, duracion);
                Thread.sleep(duracion);
                terminarEvento(tipo);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void iniciarEvento(TipoEvento tipo, long duracion) {
        // writeLock: nadie ve los flags a medio actualizar
        estado.lockEventoWrite();
        try {
            estado.activarEvento(tipo, duracion);
            aplicarEfectosInicio(tipo);
            SistemaLog.getInstance().logEventoGlobal(tipo.name() + " INICIADO");
        } finally {

```

```

        // Suelto YA el lock: el sleep de duración no debe retenerlo
        estado.unlockEventWrite();
    }
}

private void aplicarEfectosInicio(TipoEvento tipo) {
    switch (tipo) {
        case APAGON_LABORATORIO -> {
            estado.setPortalBloqueado(true);
        }
        case TORMENTA_UPSIDE_DOWN -> {
            estado.setMultiplicadorSangre(2);
        }
        case INTERVENCION_ELEVEN -> {
            estado.setDemogorgonesParalizados(true);
            // Eleven libera tantos niños como sangre haya
            int sangre = estado.getSangreTotal();
            int liberados = colmena.liberarNinos(sangre);
            estado.consumirSangre(liberados);
            SistemaLog.getInstance().log(
                "Eleven libera " + liberados + " niños " +
                "(sangre disponible: " + sangre + ")");
        }
        case RED_MENTAL -> {
            ZonaUpsideDown objetivo = zonaConMasNinos();
            estado.setZonaRedMental(objetivo);
            SistemaLog.getInstance().log(
                "Red Mental: todos los demogorgons se dirigen a " + objetivo);
        }
    }
}

private void terminarEvento(TipoEvento tipo) {
    estado.lockEventWrite();
    try {
        restaurarEfectos(tipo);
        estado.desactivarEvento();
        SistemaLog.getInstance().logEventoGlobal(tipo.name() + " FINALIZADO");
    } finally {
        estado.unlockEventWrite();
    }
}

private void restaurarEfectos(TipoEvento tipo) {
    switch (tipo) {
        case APAGON_LABORATORIO -> estado.setPortalBloqueado(false);
        case TORMENTA_UPSIDE_DOWN -> estado.setMultiplicadorSangre(1);
        case INTERVENCION_ELEVEN -> estado.setDemogorgonesParalizados(false);
        case RED_MENTAL -> estado.setZonaRedMental(null);
    }
}

private TipoEvento elegirEvento() {
    TipoEvento[] valores = TipoEvento.values();
    return valores[RNG.nextInt(valores.length)];
}

private ZonaUpsideDown zonaConMasNinos() {
    return zonas.stream()
        .max(java.util.Comparator.comparingInt(ZonaUpsideDown::contarNinos))
        .orElse(zonas.get(0));
}
}

```

// HawkinsRemote.java

```

package remote;

import java.rmi.Remote;
import java.rmi.RemoteException;

```

```
// Interfaz Remote
public interface HawkinsRemote extends Remote {

    EstadoDTO getEstado() throws RemoteException;

    void setPausado(boolean pausar) throws RemoteException;
}

```

// EstadoDTO.java

```
package remote;

import java.io.Serializable;
import java.util.List;
import java.util.Map;

// Serializable: necesario para RMI
public final class EstadoDTO implements Serializable {

    private static final long serialVersionUID = 1L;

    public final int totalNinosHawkins;

    public final Map<String, Integer> ninosPorPortal;

    public final Map<String, Integer> ninosPorZonaUD;
    public final Map<String, Integer> demosPorZonaUD;
    public final int ninosCapturadosColmena;

    public final List<String[]> rankingDemogorgons;

    public final String eventoActivo;
    public final long tiempoRestanteSeg;
    public final boolean pausado;

    public EstadoDTO(int totalNinosHawkins,
                     Map<String, Integer> ninosPorPortal,
                     Map<String, Integer> ninosPorZonaUD,
                     Map<String, Integer> demosPorZonaUD,
                     int ninosCapturadosColmena,
                     List<String[]> rankingDemogorgons,
                     String eventoActivo,
                     long tiempoRestanteSeg,
                     boolean pausado) {
        // copyOf: copias , así el cliente ve un dato real
        this.totalNinosHawkins = totalNinosHawkins;
        this.ninosPorPortal = Map.copyOf(ninosPorPortal);
        this.ninosPorZonaUD = Map.copyOf(ninosPorZonaUD);
        this.demosPorZonaUD = Map.copyOf(demosPorZonaUD);
        this.ninosCapturadosColmena = ninosCapturadosColmena;
        this.rankingDemogorgons = List.copyOf(rankingDemogorgons);
        this.eventoActivo = eventoActivo;
        this.tiempoRestanteSeg = tiempoRestanteSeg;
        this.pausado = pausado;
    }
}

```

// ServidorRemoto.java

```
package remote;

import model.Demogorgon;
import state.EstadoGlobal;
import zones.Colmena;
import zones.Portal;
import zones.ZonaHawkins;
import zones.ZonaUpsideDown;

```

```

import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

public class ServidorRemoto extends UnicastRemoteObject implements HawkinsRemote {

    private static final int    PUERTO = 1099;
    private static final String NOMBRE = "HawkinsService";

    private final ZonaHawkins    hawkins;
    private final Portal[]        portales;
    private final List<ZonaUpsideDown> zonasUD;
    private final Colmena         colmena;
    private final EstadoGlobal    estado;

    // CopyOnWrite: muchas lecturas ranking cada segundo pocas escrituras spawns
    private static final CopyOnWriteArrayList<Demogorgon> demogorgonsActivos =
        new CopyOnWriteArrayList<>();

    protected ServidorRemoto(ZonaHawkins hawkins, Portal[] portales,
                             List<ZonaUpsideDown> zonasUD, Colmena colmena,
                             EstadoGlobal estado) throws RemoteException {
        this.hawkins = hawkins;
        this.portales = portales;
        this.zonasUD = zonasUD;
        this.colmena = colmena;
        this.estado = estado;
    }

    public static void registrarDemogorgon(Demogorgon d) {
        demogorgonsActivos.add(d);
    }

    @Override
    public EstadoDTO getEstado() throws RemoteException {

        int totalHawkins = hawkins.getTotalNinos();

        Map<String, Integer> porPortal = new LinkedHashMap<>();
        for (Portal p : portales) {
            porPortal.put(p.getDestino().name(), p.getTotalEsperando());
        }

        Map<String, Integer> ninosUD = new LinkedHashMap<>();
        Map<String, Integer> demosUD = new LinkedHashMap<>();
        for (ZonaUpsideDown z : zonasUD) {
            ninosUD.put(z.getNombre().name(), z.contarNinos());
            demosUD.put(z.getNombre().name(), z.contarDemogorgons());
        }

        int capturados = colmena.getNinosCapturados();

        // Top3 por capturas
        List<String[]> ranking = demogorgonsActivos.stream()
            .sorted(Comparator.comparingInt(Demogorgon::getCapturas).reversed())
            .limit(3)
            .map(d -> new String[]{ d.getNombre(), String.valueOf(d.getCapturas()) })
            .toList();

        state.TipoEvento ev = estado.getEventoActivo();
        String eventoStr    = (ev != null) ? ev.name() : null;
        long  restanteSeg   = estado.getTiempoRestanteMs() / 1000;

        return new EstadoDTO(totalHawkins, porPortal, ninosUD, demosUD,
                              capturados, ranking, eventoStr, restanteSeg, estado.isPausado());
    }

    @Override
    public void setPausado(boolean pausar) throws RemoteException {

```

```

        if (pausar) {
            estado.pausar();
            log.SistemaLog.getInstance().log("Sistema PAUSADO por cliente remoto");
        } else {
            estado.reanudar();
            log.SistemaLog.getInstance().log("Sistema REANUDADO por cliente remoto");
        }
    }
}

public static void iniciar(ZonaHawkins hawkins, Portal[] portales,
                          List<ZonaUpsideDown> zonasUD, Colmena colmena,
                          EstadoGlobal estado) throws Exception {
    ServidorRemoto servidor = new ServidorRemoto(
        hawkins, portales, zonasUD, colmena, estado);

    Registry registry;
    try {
        // Creo el registro yo, así no hace falta lanzar rmiregistry a mano
        registry = LocateRegistry.createRegistry(PUERTO);
    } catch (RemoteException e) {
        // Si ya existía (segunda ejecución), lo cojo en vez de crearlo
        registry = LocateRegistry.getRegistry(PUERTO);
    }

    registry.rebind(NOMBRE, servidor);
    log.SistemaLog.getInstance().log(
        "Servidor RMI escuchando en puerto " + PUERTO + " → " + NOMBRE);
}
}

```

// ClienteRemoto.java

```

package remote;

public class ClienteRemoto extends javax.swing.JFrame {

    private static final int    INTERVALO_MS = 1000;
    private static final String SERVICIO     = "HawkinsService";
    private static final int    PUERTO      = 1099;

    private remote.HawkinsRemote stub;
    private final String host;
    private boolean pausadoLocal = false;

    public ClienteRemoto(String host) {
        this.host = host;
        initComponents();
        conectar();
        // Refresco automático: una llamada RMI por segundo
        new javax.swing.Timer(INTERVALO_MS, e -> tick()).start();
        setLocationRelativeTo(null);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents
    private void initComponents() {

        lblP1 = new javax.swing.JLabel();
        lblSujetos = new javax.swing.JLabel();
        lblP4 = new javax.swing.JLabel();
        lblP3 = new javax.swing.JLabel();
        lblSupC = new javax.swing.JLabel();
        lblSupL = new javax.swing.JLabel();
        lblSupB = new javax.swing.JLabel();
        lblR3 = new javax.swing.JLabel();
        lblTempo = new javax.swing.JLabel();
        lblP2 = new javax.swing.JLabel();
        lblFooter = new javax.swing.JLabel();
        lblEvL1 = new javax.swing.JLabel();
        lblColmena = new javax.swing.JLabel();
    }
}

```



```

lblSupA = new javax.swing.JLabel();
lblR1 = new javax.swing.JLabel();
lblOtroA = new javax.swing.JLabel();
lblOtroC = new javax.swing.JLabel();
lblOtroL = new javax.swing.JLabel();
lblOtroB = new javax.swing.JLabel();
lblR2 = new javax.swing.JLabel();
lblEvL2 = new javax.swing.JLabel();
lblEvL3 = new javax.swing.JLabel();
btnPulsador = new javax.swing.JButton();
lblBg = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setMinimumSize(new java.awt.Dimension(1280, 751));
getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

lblP1.setBackground(new java.awt.Color(205, 198, 152));
lblP1.setFont(new java.awt.Font("Monospaced", 1, 18));
lblP1.setForeground(new java.awt.Color(51, 51, 51));
lblP1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP1.setOpaque(true);
getContentPane().add(lblP1, new org.netbeans.lib.awtextra.AbsoluteConstraints(190, 390, 50, 35));

lblSujetos.setBackground(new java.awt.Color(34, 68, 41));
lblSujetos.setFont(new java.awt.Font("Monospaced", 1, 24));
lblSujetos.setForeground(new java.awt.Color(255, 0, 0));
lblSujetos.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSujetos.setOpaque(true);
getContentPane().add(lblSujetos, new org.netbeans.lib.awtextra.AbsoluteConstraints(240, 230, 50, 40));

lblP4.setBackground(new java.awt.Color(205, 198, 152));
lblP4.setFont(new java.awt.Font("Monospaced", 1, 18));
lblP4.setForeground(new java.awt.Color(51, 51, 51));
lblP4.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP4.setOpaque(true);
getContentPane().add(lblP4, new org.netbeans.lib.awtextra.AbsoluteConstraints(190, 600, 50, 35));

lblP3.setBackground(new java.awt.Color(205, 198, 152));
lblP3.setFont(new java.awt.Font("Monospaced", 1, 18));
lblP3.setForeground(new java.awt.Color(51, 51, 51));
lblP3.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP3.setOpaque(true);
getContentPane().add(lblP3, new org.netbeans.lib.awtextra.AbsoluteConstraints(190, 530, 50, 35));

lblSupC.setBackground(new java.awt.Color(202, 250, 198));
lblSupC.setFont(new java.awt.Font("Monospaced", 1, 18));
lblSupC.setForeground(new java.awt.Color(51, 51, 51));
lblSupC.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSupC.setOpaque(true);
getContentPane().add(lblSupC, new org.netbeans.lib.awtextra.AbsoluteConstraints(590, 420, 50, 40));

lblSupL.setBackground(new java.awt.Color(202, 250, 198));
lblSupL.setFont(new java.awt.Font("Monospaced", 1, 18));
lblSupL.setForeground(new java.awt.Color(51, 51, 51));
lblSupL.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSupL.setOpaque(true);
getContentPane().add(lblSupL, new org.netbeans.lib.awtextra.AbsoluteConstraints(670, 320, 50, 40));

lblSupB.setBackground(new java.awt.Color(202, 250, 198));
lblSupB.setFont(new java.awt.Font("Monospaced", 1, 18));
lblSupB.setForeground(new java.awt.Color(51, 51, 51));
lblSupB.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSupB.setOpaque(true);
getContentPane().add(lblSupB, new org.netbeans.lib.awtextra.AbsoluteConstraints(640, 250, 50, 60));

lblR3.setBackground(new java.awt.Color(196, 198, 176));
lblR3.setFont(new java.awt.Font("Monospaced", 1, 18));
lblR3.setForeground(new java.awt.Color(51, 51, 51));
lblR3.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblR3.setOpaque(true);
getContentPane().add(lblR3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 330, 55, 40));

```

```

lblTempo.setBackground(new java.awt.Color(7, 17, 9));
lblTempo.setFont(new java.awt.Font("Monospaced", 1, 13));
lblTempo.setForeground(new java.awt.Color(150, 250, 150));
lblTempo.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblTempo.setOpaque(true);
getContentPane().add(lblTempo, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 610, 90, 30));

lblP2.setBackground(new java.awt.Color(205, 198, 152));
lblP2.setFont(new java.awt.Font("Monospaced", 1, 18));
lblP2.setForeground(new java.awt.Color(51, 51, 51));
lblP2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP2.setOpaque(true);
getContentPane().add(lblP2, new org.netbeans.lib.awtextra.AbsoluteConstraints(190, 460, 50, 35));

lblFooter.setBackground(new java.awt.Color(108, 103, 83));
lblFooter.setFont(new java.awt.Font("Monospaced", 1, 13));
lblFooter.setForeground(new java.awt.Color(51, 51, 51));
lblFooter.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblFooter.setOpaque(true);
getContentPane().add(lblFooter, new org.netbeans.lib.awtextra.AbsoluteConstraints(680, 720, 380, 30));

lblEvL1.setBackground(new java.awt.Color(14, 28, 16));
lblEvL1.setFont(new java.awt.Font("Monospaced", 1, 13));
lblEvL1.setForeground(new java.awt.Color(150, 250, 150));
lblEvL1.setOpaque(true);
getContentPane().add(lblEvL1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1130, 490, 140, 30));

lblColmena.setBackground(new java.awt.Color(209, 252, 198));
lblColmena.setFont(new java.awt.Font("Monospaced", 1, 18));
lblColmena.setForeground(new java.awt.Color(153, 0, 0));
lblColmena.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblColmena.setOpaque(true);
getContentPane().add(lblColmena, new org.netbeans.lib.awtextra.AbsoluteConstraints(860, 540, 45, 35));

lblSupA.setBackground(new java.awt.Color(202, 250, 198));
lblSupA.setFont(new java.awt.Font("Monospaced", 1, 18));
lblSupA.setForeground(new java.awt.Color(51, 51, 51));
lblSupA.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSupA.setOpaque(true);
getContentPane().add(lblSupA, new org.netbeans.lib.awtextra.AbsoluteConstraints(690, 460, 50, 50));

lblR1.setBackground(new java.awt.Color(196, 198, 176));
lblR1.setFont(new java.awt.Font("Monospaced", 1, 18));
lblR1.setForeground(new java.awt.Color(51, 51, 51));
lblR1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblR1.setOpaque(true);
getContentPane().add(lblR1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 210, 55, 40));

lblOtroA.setBackground(new java.awt.Color(174, 225, 168));
lblOtroA.setFont(new java.awt.Font("Monospaced", 1, 18));
lblOtroA.setForeground(new java.awt.Color(51, 51, 51));
lblOtroA.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblOtroA.setOpaque(true);
getContentPane().add(lblOtroA, new org.netbeans.lib.awtextra.AbsoluteConstraints(990, 470, 45, 30));

lblOtroC.setBackground(new java.awt.Color(174, 225, 168));
lblOtroC.setFont(new java.awt.Font("Monospaced", 1, 18));
lblOtroC.setForeground(new java.awt.Color(51, 51, 51));
lblOtroC.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblOtroC.setOpaque(true);
getContentPane().add(lblOtroC, new org.netbeans.lib.awtextra.AbsoluteConstraints(880, 415, 60, 45));

lblOtroL.setBackground(new java.awt.Color(174, 225, 168));
lblOtroL.setFont(new java.awt.Font("Monospaced", 1, 18));
lblOtroL.setForeground(new java.awt.Color(51, 51, 51));
lblOtroL.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblOtroL.setOpaque(true);
getContentPane().add(lblOtroL, new org.netbeans.lib.awtextra.AbsoluteConstraints(950, 310, 80, 60));

lblOtroB.setBackground(new java.awt.Color(174, 225, 168));
lblOtroB.setFont(new java.awt.Font("Monospaced", 1, 18));
lblOtroB.setForeground(new java.awt.Color(51, 51, 51));

```

```

lblOtroB.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblOtroB.setOpaque(true);
getContentPane().add(lblOtroB, new org.netbeans.lib.awtextra.AbsoluteConstraints(930, 255, 50, 45));

lblR2.setBackground(new java.awt.Color(196, 198, 176));
lblR2.setFont(new java.awt.Font("Monospaced", 1, 18));
lblR2.setForeground(new java.awt.Color(51, 51, 51));
lblR2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblR2.setOpaque(true);
getContentPane().add(lblR2, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 270, 55, 40));

lblEvL2.setBackground(new java.awt.Color(14, 28, 16));
lblEvL2.setFont(new java.awt.Font("Monospaced", 1, 13));
lblEvL2.setForeground(new java.awt.Color(150, 250, 150));
lblEvL2.setOpaque(true);
getContentPane().add(lblEvL2, new org.netbeans.lib.awtextra.AbsoluteConstraints(1090, 520, 170, 30));

lblEvL3.setBackground(new java.awt.Color(14, 28, 16));
lblEvL3.setFont(new java.awt.Font("Monospaced", 1, 13));
lblEvL3.setForeground(new java.awt.Color(150, 250, 150));
lblEvL3.setOpaque(true);
getContentPane().add(lblEvL3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1110, 550, 130, 30));

btnPulsador.setToolTipText("PAUSAR / REANUDAR la simulación remota");
btnPulsador.setBorderPainted(false);
btnPulsador.setContentAreaFilled(false);
btnPulsador.addActionListener(this::btnPulsadorActionPerformed);
getContentPane().add(btnPulsador, new org.netbeans.lib.awtextra.AbsoluteConstraints(490, 610, 160, 110));

lblBg.setIcon(new javax.swing.ImageIcon(getClass().getResource("/remote/img/consola.png")));
getContentPane().add(lblBg, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, -1, -1));

pack();
} // </editor-fold> // GEN-END: initComponents

// Pausa en el servidor
private void btnPulsadorActionPerformed(java.awt.event.ActionEvent evt) { // GEN-FIRST:event_btnPulsadorActionPerformed
    if (stub == null) return;
    try {
        pausadoLocal = !pausadoLocal;
        stub.setPausado(pausadoLocal);
    } catch (Exception ex) {
        setTitle("Error pulsador: " + ex.getMessage());
    }
} // GEN-LAST:event_btnPulsadorActionPerformed

// Busco el servicio en el registro RMI
private void conectar() {
    try {
        java.rmi.registry.Registry reg = java.rmi.registry.LocateRegistry.getRegistry(host, PUERTO);
        stub = (remote.HawkinsRemote) reg.lookup(SERVICIO);
        setTitle("Cliente RMI - Conectado a " + host + ":" + PUERTO);
    } catch (Exception e) {
        setTitle("Cliente RMI - Error de conexión: " + e.getMessage());
    }
}

// Cada segundo: pido el estado al servidor y refresco la GUI
private void tick() {
    if (stub == null) { conectar(); return; }
    try { actualizar(stub.getEstado()); }
    catch (Exception ex) {
        setTitle("Cliente RMI - Conexión perdida: " + ex.getMessage());
        stub = null;
    }
}

private void actualizar(remote.EstadoDTO dto) {
    lblSujetos.setText "[" + dto.totalNinosHawkins + "]";

    String[] keys = {"BOSQUE", "LABORATORIO", "CENTRO_COMERCIAL", "ALCANTARILLADO"};
    int p1 = dto.ninosPorPortal.getDefault(keys[0], 0);

```

```

int p2 = dto.ninosPorPortal.getOrDefault(keys[1], 0);
int p3 = dto.ninosPorPortal.getOrDefault(keys[2], 0);
int p4 = dto.ninosPorPortal.getOrDefault(keys[3], 0);
lblP1.setText "[" + p1 + "]";
lblP2.setText "[" + p2 + "]";
lblP3.setText "[" + p3 + "]";
lblP4.setText "[" + p4 + "]";

lblSupB.setText "(" + p1 + ")";
lblSupL.setText "(" + p2 + ")";
lblSupC.setText "(" + p3 + ")";
lblSupA.setText "(" + p4 + ")";

lblOtroB.setText "(" + dto.ninosPorZonaUD.getOrDefault(keys[0], 0) + ")";
lblOtroL.setText "(" + dto.ninosPorZonaUD.getOrDefault(keys[1], 0) + ")";
lblOtroC.setText "(" + dto.ninosPorZonaUD.getOrDefault(keys[2], 0) + ")";
lblOtroA.setText "(" + dto.ninosPorZonaUD.getOrDefault(keys[3], 0) + ")";

lblColmena.setText "[" + dto.ninosCapturadosColmena + "]";

setRank(lblR1, dto, 0);
setRank(lblR2, dto, 1);
setRank(lblR3, dto, 2);

if (dto.eventoActivo == null) {
    lblEvL1.setText("Sin evento"); lblEvL2.setText("activo"); lblEvL3.setText("");
    lblTempo.setText("--:--");
} else {
    String[] d = describir(dto.eventoActivo);
    lblEvL1.setText(d[0]); lblEvL2.setText(d[1]); lblEvL3.setText(d[2]);
    lblTempo.setText(String.format("00:00ds", dto.tiempoRestanteSeg));
}
lblFooter.setText(" " + java.time.LocalDateTime.now()
    .format(java.time.format.DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss"))
    + " - CLIENTE REMOTO ");
pausadoLocal = dto.pausado;
}

private void setRank(javax.swing.JLabel l, remote.EstadoDTO dto, int idx) {
    l.setText(idx >= dto.rankingDemogorgons.size() ? "(--)"
        : "(" + dto.rankingDemogorgons.get(idx)[1] + ")");
}

// Texto legible para cada tipo de evento
private static String[] describir(String tipo) {
    switch (tipo) {
        case "APAGON_LABORATORIO": return new String[]{"TIPO: Interrupcion", "Total de Energia", "(APAGON LAB)"};
        case "TORMENTA_UPSIDE_DOWN": return new String[]{"TIPO: Tormenta", "del Upside Down", ""};
        case "INTERVENCION_ELEVEN": return new String[]{"TIPO: Intervencion", "de Eleven", ""};
        case "RED_MENTAL": return new String[]{"TIPO: La Red", "Mental (Mindflayer)", ""};
        default: return new String[]{tipo, "", ""};
    }
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton btnPulsador;
private javax.swing.JLabel lblBg;
private javax.swing.JLabel lblColmena;
private javax.swing.JLabel lblEvL1;
private javax.swing.JLabel lblEvL2;
private javax.swing.JLabel lblEvL3;
private javax.swing.JLabel lblFooter;
private javax.swing.JLabel lblOtroA;
private javax.swing.JLabel lblOtroB;
private javax.swing.JLabel lblOtroC;
private javax.swing.JLabel lblOtroL;
private javax.swing.JLabel lblP1;
private javax.swing.JLabel lblP2;
private javax.swing.JLabel lblP3;
private javax.swing.JLabel lblP4;
private javax.swing.JLabel lblR1;
private javax.swing.JLabel lblR2;

```

```

private javax.swing.JLabel lblR3;
private javax.swing.JLabel lblSujetos;
private javax.swing.JLabel lblSupA;
private javax.swing.JLabel lblSupB;
private javax.swing.JLabel lblSupC;
private javax.swing.JLabel lblSupL;
private javax.swing.JLabel lblTempo;
// End of variables declaration//GEN-END:variables
}

```

// InicioGUI.java

```

package gui;

import javax.imageio.ImageIO;
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.image.BufferedImage;
import java.io.IOException;
import java.io.InputStream;

public class InicioGUI extends javax.swing.JFrame {

    private static final String IMG_RES = "/gui/img/inicio.png";
    private static final int    IMG_W  = 1280;
    private static final int    IMG_H  = 740;
    // Coords del botón START dentro de la imagen
    private static final Rectangle BTN_BOUNDS = new Rectangle(388, 410, 460, 72);

    private boolean pressed = false;
    private final Runnable onStart;

    public InicioGUI(Runnable onStart) {
        this.onStart = onStart;
        initComponents();
        setupContent();
        pack();
        setLocationRelativeTo(null);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("HAWKINS LAB - SISTEMA DE MONITOREO");
        setResizable(false);

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGap(0, IMG_W, Short.MAX_VALUE)
                )
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGap(0, IMG_H, Short.MAX_VALUE)
                )
        );

        pack();
    } // </editor-fold>

    private void setupContent() {
        BufferedImage bg = cargarImagen();
        InicioPanel panel = new InicioPanel(bg);
        panel.setPreferredSize(new Dimension(IMG_W, IMG_H));
        setContentPane(panel);
    }
}

```

```

// Primero pruebo desde el classpath, si falla busco en disco. Último recurso: rectángulo verde
private BufferedImage cargarImagen() {
    try (InputStream in = getClass().getResourceAsStream(IMG_RES)) {
        if (in != null) return ImageIO.read(in);
    } catch (IOException ignored) {}
    try {
        return ImageIO.read(new java.io.File("src/gui/img/inicio.png"));
    } catch (IOException ex) {
        BufferedImage ph = new BufferedImage(IMG_W, IMG_H, BufferedImage.TYPE_INT_RGB);
        Graphics2D g = ph.createGraphics();
        g.setColor(new Color(8, 18, 10));
        g.fillRect(0, 0, IMG_W, IMG_H);
        g.dispose();
        return ph;
    }
}

private class InicioPanel extends JPanel {
    private final BufferedImage bg;
    private boolean hover = false;

    InicioPanel(BufferedImage bg) {
        this.bg = bg;
        setPreferredSize(new Dimension(bg.getWidth(), bg.getHeight()));
        setLayout(null);

        addMouseMotionListener(new MouseAdapter() {
            @Override public void mouseMoved(MouseEvent e) {
                boolean h = BTN_BOUNDS.contains(e.getPoint());
                if (h != hover) { hover = h; repaint(); }
                setCursor(h ? Cursor.getPredefinedCursor(Cursor.HAND_CURSOR)
                    : Cursor.getDefaultCursor());
            }
        });

        addMouseListener(new MouseAdapter() {
            @Override public void mousePressed(MouseEvent e) {
                if (BTN_BOUNDS.contains(e.getPoint())) { pressed = true; repaint(); }
            }
            @Override public void mouseReleased(MouseEvent e) {
                if (pressed && BTN_BOUNDS.contains(e.getPoint())) {
                    pressed = false; repaint();
                    // Pequeño delay para que se vea la animación de pulsado
                    Timer t = new Timer(180, ev -> {
                        ((Timer) ev.getSource()).stop();
                        dispose();
                        if (onStart != null) onStart.run();
                    });
                    t.setRepeats(false); t.start();
                } else { pressed = false; repaint(); }
            }
            @Override public void mouseExited(MouseEvent e) {
                if (hover) { hover = false; repaint(); }
            }
        });

        // Atajo: ENTER también arranca el sistema
        KeyStroke ks = KeyStroke.getKeyStroke("ENTER");
        getInputMap(WHEN_IN_FOCUSED_WINDOW).put(ks, "start");
        getActionMap().put("start", new AbstractAction() {
            @Override public void actionPerformed(java.awt.event.ActionEvent e) {
                pressed = true; repaint();
                Timer t = new Timer(220, ev -> {
                    ((Timer) ev.getSource()).stop();
                    pressed = false;
                    dispose();
                    if (onStart != null) onStart.run();
                });
                t.setRepeats(false); t.start();
            }
        });
    }
}

```

```

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
    g2.drawImage(bg, 0, 0, getWidth(), getHeight(), null);

    // Overlay según estado del botón: pulsado oscurece, hover ilumina
    if (pressed) {
        g2.setColor(new Color(0, 0, 0, 110));
        g2.fillRect(BTN_BOUNDS.x + 2, BTN_BOUNDS.y + 3,
            BTN_BOUNDS.width, BTN_BOUNDS.height);
        g2.setColor(new Color(255, 255, 255, 35));
        g2.fillRect(BTN_BOUNDS.x, BTN_BOUNDS.y,
            BTN_BOUNDS.width, BTN_BOUNDS.height);
    } else if (hover) {
        g2.setColor(new Color(255, 255, 255, 50));
        g2.fillRect(BTN_BOUNDS.x, BTN_BOUNDS.y,
            BTN_BOUNDS.width, BTN_BOUNDS.height);
        g2.setColor(new Color(80, 255, 120, 180));
        g2.setStroke(new BasicStroke(2.0f));
        g2.drawRect(BTN_BOUNDS.x - 1, BTN_BOUNDS.y - 1,
            BTN_BOUNDS.width + 2, BTN_BOUNDS.height + 2);
    }
    g2.dispose();
}
}
}

```

// HawkinsGUI.java

```

package gui;

public class HawkinsGUI extends javax.swing.JFrame {

    private final zones.ZonaHawkins hawkins;
    private final zones.Portal[] portales;
    private final java.util.List<zones.ZonaUpsideDown> zonasUD;
    private final zones.Colmena colmena;
    private final state.EstadoGlobal estado;

    public HawkinsGUI(zones.ZonaHawkins hawkins, zones.Portal[] portales,
        java.util.List<zones.ZonaUpsideDown> zonasUD,
        zones.Colmena colmena, state.EstadoGlobal estado) {
        this.hawkins = hawkins;
        this.portales = portales;
        this.zonasUD = zonasUD;
        this.colmena = colmena;
        this.estado = estado;
        initComponents();
        // Refresco cada 500ms desde el EDT
        new javax.swing.Timer(500, e -> refresh()).start();
        setLocationRelativeTo(null);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents
    private void initComponents() {

        lblSotano = new javax.swing.JLabel();
        lblCalleP = new javax.swing.JLabel();
        lblSangre = new javax.swing.JLabel();
        lblP4B = new javax.swing.JLabel();
        lblP4A = new javax.swing.JLabel();
        lblP3B = new javax.swing.JLabel();
        lblP3A = new javax.swing.JLabel();
        lblP2B = new javax.swing.JLabel();
        lblP2A = new javax.swing.JLabel();
    }
}

```

```

lblP1B = new javax.swing.JLabel();
lblP1A = new javax.swing.JLabel();
lblRadio = new javax.swing.JLabel();
lblFooter = new javax.swing.JLabel();
lblEvento = new javax.swing.JLabel();
lblColmena = new javax.swing.JLabel();
lblUDAD = new javax.swing.JLabel();
lblUDAN = new javax.swing.JLabel();
lblUDCD = new javax.swing.JLabel();
lblUDCN = new javax.swing.JLabel();
lblUDLD = new javax.swing.JLabel();
lblUDLN = new javax.swing.JLabel();
lblUDBD = new javax.swing.JLabel();
lblUDBN = new javax.swing.JLabel();
lblBg = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setMaximumSize(new java.awt.Dimension(1280, 751));
setMinimumSize(new java.awt.Dimension(1280, 751));
getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

lblSotano.setBackground(new java.awt.Color(34, 68, 41));
lblSotano.setFont(new java.awt.Font("Monospaced", 1, 18)); // NOI18N
lblSotano.setForeground(new java.awt.Color(150, 250, 150));
lblSotano.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSotano.setOpaque(true);
getContentPane().add(lblSotano, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 360, 280, 80));

lblCalleP.setBackground(new java.awt.Color(34, 68, 41));
lblCalleP.setFont(new java.awt.Font("Monospaced", 1, 18)); // NOI18N
lblCalleP.setForeground(new java.awt.Color(150, 250, 150));
lblCalleP.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblCalleP.setOpaque(true);
getContentPane().add(lblCalleP, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 190, 280, 90));

lblSangre.setBackground(new java.awt.Color(56, 34, 21));
lblSangre.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblSangre.setForeground(new java.awt.Color(153, 0, 0));
lblSangre.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblSangre.setOpaque(true);
getContentPane().add(lblSangre, new org.netbeans.lib.awtextra.AbsoluteConstraints(360, 590, 50, 30));

lblP4B.setBackground(new java.awt.Color(25, 58, 29));
lblP4B.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP4B.setForeground(new java.awt.Color(150, 250, 150));
lblP4B.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP4B.setOpaque(true);
getContentPane().add(lblP4B, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 560, 220, 60));

lblP4A.setBackground(new java.awt.Color(34, 68, 41));
lblP4A.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP4A.setForeground(new java.awt.Color(150, 250, 150));
lblP4A.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP4A.setOpaque(true);
getContentPane().add(lblP4A, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 510, 220, 50));

lblP3B.setBackground(new java.awt.Color(25, 58, 29));
lblP3B.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP3B.setForeground(new java.awt.Color(150, 250, 150));
lblP3B.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP3B.setOpaque(true);
getContentPane().add(lblP3B, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 440, 220, 50));

lblP3A.setBackground(new java.awt.Color(34, 68, 41));
lblP3A.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP3A.setForeground(new java.awt.Color(150, 250, 150));
lblP3A.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP3A.setOpaque(true);
getContentPane().add(lblP3A, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 390, 220, 50));

lblP2B.setBackground(new java.awt.Color(25, 58, 29));
lblP2B.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N

```



```

lblP2B.setForeground(new java.awt.Color(150, 250, 150));
lblP2B.setFont(new javax.swing.SwingConstants.CENTER);
lblP2B.setOpaque(true);
getContentPane().add(lblP2B, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 320, 220, 60));

lblP2A.setBackground(new java.awt.Color(34, 68, 41));
lblP2A.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP2A.setForeground(new java.awt.Color(150, 250, 150));
lblP2A.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP2A.setOpaque(true);
getContentPane().add(lblP2A, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 260, 220, 60));

lblP1B.setBackground(new java.awt.Color(25, 58, 29));
lblP1B.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP1B.setForeground(new java.awt.Color(150, 250, 150));
lblP1B.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP1B.setOpaque(true);
getContentPane().add(lblP1B, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 190, 220, 60));

lblP1A.setBackground(new java.awt.Color(34, 68, 41));
lblP1A.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblP1A.setForeground(new java.awt.Color(150, 250, 150));
lblP1A.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblP1A.setOpaque(true);
getContentPane().add(lblP1A, new org.netbeans.lib.awtextra.AbsoluteConstraints(500, 130, 220, 60));

lblRadio.setBackground(new java.awt.Color(34, 68, 41));
lblRadio.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblRadio.setForeground(new java.awt.Color(150, 250, 150));
lblRadio.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblRadio.setOpaque(true);
getContentPane().add(lblRadio, new org.netbeans.lib.awtextra.AbsoluteConstraints(240, 560, 80, 60));

lblFooter.setBackground(new java.awt.Color(108, 103, 83));
lblFooter.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblFooter.setForeground(new java.awt.Color(51, 51, 51));
lblFooter.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblFooter.setOpaque(true);
getContentPane().add(lblFooter, new org.netbeans.lib.awtextra.AbsoluteConstraints(410, 660, 420, 30));

lblEvento.setBackground(new java.awt.Color(14, 28, 16));
lblEvento.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblEvento.setForeground(new java.awt.Color(150, 250, 150));
lblEvento.setOpaque(true);
getContentPane().add(lblEvento, new org.netbeans.lib.awtextra.AbsoluteConstraints(410, 70, 550, 30));

lblColmena.setBackground(new java.awt.Color(56, 34, 21));
lblColmena.setFont(new java.awt.Font("Monospaced", 1, 36)); // NOI18N
lblColmena.setForeground(new java.awt.Color(153, 0, 0));
lblColmena.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblColmena.setOpaque(true);
getContentPane().add(lblColmena, new org.netbeans.lib.awtextra.AbsoluteConstraints(980, 210, 90, 60));

lblUDAD.setBackground(new java.awt.Color(25, 58, 29));
lblUDAD.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDAD.setForeground(new java.awt.Color(150, 250, 150));
lblUDAD.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDAD.setOpaque(true);
getContentPane().add(lblUDAD, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 550, 180, 50));

lblUDAN.setBackground(new java.awt.Color(34, 68, 41));
lblUDAN.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDAN.setForeground(new java.awt.Color(150, 250, 150));
lblUDAN.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDAN.setOpaque(true);
getContentPane().add(lblUDAN, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 500, 180, 50));

lblUDCD.setBackground(new java.awt.Color(25, 58, 29));
lblUDCD.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDCD.setForeground(new java.awt.Color(150, 250, 150));
lblUDCD.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDCD.setOpaque(true);

```

```

getContentPane().add(lblUDCD, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 440, 180, 50));

lblUDCN.setBackground(new java.awt.Color(34, 68, 41));
lblUDCN.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDCN.setForeground(new java.awt.Color(150, 250, 150));
lblUDCN.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDCN.setOpaque(true);
getContentPane().add(lblUDCN, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 380, 180, 60));

lblUDLD.setBackground(new java.awt.Color(25, 58, 29));
lblUDLD.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDLD.setForeground(new java.awt.Color(150, 250, 150));
lblUDLD.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDLD.setOpaque(true);
getContentPane().add(lblUDLD, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 310, 190, 50));

lblUDLN.setBackground(new java.awt.Color(34, 68, 41));
lblUDLN.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDLN.setForeground(new java.awt.Color(150, 250, 150));
lblUDLN.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDLN.setOpaque(true);
getContentPane().add(lblUDLN, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 260, 190, 50));

lblUDBD.setBackground(new java.awt.Color(25, 58, 29));
lblUDBD.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDBD.setForeground(new java.awt.Color(150, 250, 150));
lblUDBD.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDBD.setOpaque(true);
getContentPane().add(lblUDBD, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 190, 190, 60));

lblUDBN.setBackground(new java.awt.Color(34, 68, 41));
lblUDBN.setFont(new java.awt.Font("Monospaced", 1, 13)); // NOI18N
lblUDBN.setForeground(new java.awt.Color(150, 250, 150));
lblUDBN.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblUDBN.setOpaque(true);
getContentPane().add(lblUDBN, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 130, 190, 60));

lblBg.setIcon(new javax.swing.ImageIcon(getClass().getResource("/gui/img/hawkins_server.png"))); // NOI18N
getContentPane().add(lblBg, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, -1, -1));

pack();
} // </editor-fold> // GEN-END: initComponents

// Lo dispara el Timer 2 veces por segundo: vuelca el modelo a los labels
private void refresh() {
    lblCalleP.setText(join(hawkins.getCallePrincipalIds()));
    lblSotano.setText(join(hawkins.getSotanoIds()));
    lblRadio.setText(join(hawkins.getRadioIds()));
    lblSangre.setText(String.valueOf(estado.getSangreTotal()));

    javax.swing.JLabel[] pa = {lblP1A, lblP2A, lblP3A, lblP4A};
    javax.swing.JLabel[] pb = {lblP1B, lblP2B, lblP3B, lblP4B};
    for (int i = 0; i < 4; i++) {
        int n = portales[i].getTotalEsperando();
        pa[i].setText(portales[i].getDestino().name() + " [" + n + "]");
        pb[i].setText(n > 0 ? "esperando" : "-");
    }
    javax.swing.JLabel[] un = {lblUDBN, lblUDLN, lblUDCN, lblUDAN};
    javax.swing.JLabel[] ud = {lblUDBD, lblUDLD, lblUDCD, lblUDAD};
    for (int i = 0; i < 4; i++) {
        un[i].setText(join(zonasUD.get(i).getNinoIds()));
        ud[i].setText(join(zonasUD.get(i).getDemogorgonIds()));
    }
    lblColmena.setText(String.valueOf(colmena.getNinosCapturados()));

    state.TipoEvento ev = estado.getEventoActivo();
    lblEvento.setText(ev == null ? " EVENTO: - "
        : " EVENTO: " + ev.name() + " (" + (estado.getTiempoRestanteMs()/1000) + "s) ");
    lblFooter.setText(" " + java.time.LocalDateTime.now()
        .format(java.time.format.DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss"))
        + " - OPERADOR H.E. ");
}

```

```

// Si hay más de 3 IDs, trunco con "+N" para no desbordar el label
private static String join(java.util.List<String> l) {
    if (l.isEmpty()) return "-";
    return l.size() <= 3 ? String.join(", ", l)
        : String.join(", ", l.subList(0, 3)) + " +" + (l.size()-3);
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JLabel lblBg;
private javax.swing.JLabel lblCalleP;
private javax.swing.JLabel lblColmena;
private javax.swing.JLabel lblEvento;
private javax.swing.JLabel lblFooter;
private javax.swing.JLabel lblP1A;
private javax.swing.JLabel lblP1B;
private javax.swing.JLabel lblP2A;
private javax.swing.JLabel lblP2B;
private javax.swing.JLabel lblP3A;
private javax.swing.JLabel lblP3B;
private javax.swing.JLabel lblP4A;
private javax.swing.JLabel lblP4B;
private javax.swing.JLabel lblRadio;
private javax.swing.JLabel lblSangre;
private javax.swing.JLabel lblSotano;
private javax.swing.JLabel lblUDAD;
private javax.swing.JLabel lblUDAN;
private javax.swing.JLabel lblUDBD;
private javax.swing.JLabel lblUDBN;
private javax.swing.JLabel lblUDCD;
private javax.swing.JLabel lblUDCN;
private javax.swing.JLabel lblUDLD;
private javax.swing.JLabel lblUDLN;
// End of variables declaration//GEN-END:variables
}

```